

RLpatch: A Robust Low-Overhead Website Fingerprinting Defense Method Based on Reinforcement Learning Within Sensitive Regions

Shuangwu Chen^{ID}, *Member, IEEE*, Siyang Chen^{ID}, Yuxing Wei, Dong Jin^{ID}, Xiaobin Tan^{ID}, *Member, IEEE*, Xiaofeng Jiang, and Jian Yang^{ID}, *Senior Member, IEEE*

Abstract—Website Fingerprinting (WF) attacks have posed a serious threat to the anonymity of the onion router (Tor) communication system, as attackers can passively pry into the encrypted traffic and infer the website visited by users. To defend against WF, recent studies focus on adversarial perturbations. However, most of them suffer from a high bandwidth overhead and a low defense performance. To address this problem, our basic idea is to generate perturbation only on the sensitive regions, which can effectively mask the website's fingerprint, thus misleading the WF attack models and reducing the bandwidth overhead. In this paper, we formulate a joint optimization problem of perturbation position and magnitude by confining the perturbations within sensitive regions, which is rarely considered in the literature. We propose a robust low-overhead WF defense method based on reinforcement learning (RL), named RLpatch. RLpatch identifies the common sensitive regions of various surrogate models and adjusts perturbation according to the query result from a query WF model. It further employs the positional frequency of perturbations to generate a common perturbation paradigm for different traces of a same website. Experimental results show that RLpatch achieves higher defense performance, lower bandwidth overhead and better robustness against adversarial training compared to the state-of-the-art methods.

Index Terms—Traffic analysis, website fingerprinting, reinforcement learning, adversarial defense.

I. INTRODUCTION

TO MITIGATE the threats of online monitoring and examination, an increasing number of Internet users are relying on using privacy-enhancing technologies (PETs) [1], such as

Tor [2] and VPN [3], to encrypt the transmission content and conceal communication relationships when browsing the website. Although these PETs reduce privacy threats to some extent, a local attacker can still pry into which website the user is visiting via the website fingerprinting (WF) attack [4], which poses a significant challenge to the anonymity of users' browsing activities. Recently, Deep Neural Networks (DNNs) based WF attack models have achieved remarkable effectiveness in website identification [5]. It is necessary to develop an effective defense method against DNN-based WF attacks.

Previous defenses are mainly divided into obfuscation and regularization defenses. Obfuscation defenses mislead attackers by randomly injecting dummy packets and padding time delay [6], [7]. Regularization defenses interfere with attackers by shaping traffic traces into a fixed pattern to conceal traffic characteristics [8]. However, these defenses induce a high bandwidth overhead and introduce extra time delay. Recently, adversarial learning based WF defenses have revealed that adding tiny adversarial perturbations to the original traffic traces could confuse the DNN-based WF attack models [4], [9]. Compared to the previous defenses, they achieve a high defense performance and a low bandwidth overhead. To achieve real-time defense, some recent adversarial learning based WF defenses like [10] dynamically determine packet injection based on each ongoing packet. However, they need to determine the perturbations in time, which may degrade the user's browsing experience. Moreover, existing adversarial learning based WF defenses have rarely considered the generalization capability of the defense model. As a result, these defenses may be effective against the known WF attack models used for training, but may struggle to maintain the performance when facing unseen WF attack models.

To overcome the existing drawbacks, we aim to develop an effective WF defense method that can achieve real-time defense with a low bandwidth overhead but also ensure strong generalization capability across different WF attack models. We have an insight that WF attack models tend to pay more attention to some sensitive regions of a traffic trace for website identification, which often represent the distinctive fingerprints of a website. By introducing perturbations in the common sensitive regions of multiple WF attack models, it is possible to reduce overall bandwidth overhead and

Received 10 February 2025; revised 24 June 2025 and 17 August 2025; accepted 22 August 2025. Date of publication 26 August 2025; date of current version 5 December 2025. This work was supported by the National Natural Science Foundation of China (Grant No. U23A20275) and the Open Project Program of Guangxi Key Laboratory of Digital Infrastructure (Grant No. GXDIOP2024001). The associate editor coordinating the review of this article and approving it for publication was P. Yu. (*Corresponding authors: Dong Jin; Jian Yang.*)

Shuangwu Chen, Siyang Chen, Xiaobin Tan, Xiaofeng Jiang, and Jian Yang are with the Department of Automation, University of Science and Technology of China, Hefei 230026, China (e-mail: chensw@ustc.edu.cn; csy1366@mail.ustc.edu.cn; xbtan@ustc.edu.cn; jxf@ustc.edu.cn; jianyang@ustc.edu.cn).

Yuxing Wei is with the Guangxi Key Laboratory of Digital Infrastructure, Guangxi Zhuang Autonomous Region Information Center, Nanning 530028, China (e-mail: weiyustar@163.com).

Dong Jin is with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei, 230088, China (e-mail: kingdon@ustc.edu.cn).

Digital Object Identifier 10.1109/TNSM.2025.3602964

deceive unseen models. We can further offline train a common perturbation paradigm specifically tailored for each website and apply the perturbation to all the live traffic of the website.

In light of the above idea, we propose a robust low-overhead WF defense method, named RLpatch. By analyzing the impact of different regions of traffic traces on the WF attack models' decisions, RLpatch can pinpoint the sensitive regions. Afterwards, RLpatch employs reinforcement learning (RL) to adjust the positions and magnitude of the adversarial perturbations according to the query results from a query model. Rather than querying indiscriminately all positions of the entire trace, RLpatch focuses its query and generate perturbations just on the sensitive regions. To enhance the robustness against adversarial training, we introduce a disturbance factor, which is unknown to the attackers, into the reward to increase the diversity of the perturbations. Finally, RLpatch aggregates multiple perturbations of a website into a common perturbation paradigm, which serves as a defense strategy for all traffic traces of the corresponding website. Based on the injection positions and the number of dummy packets specified by the perturbation paradigm, RLpatch can achieve real-time defense for the traffic traces of the corresponding website.

- We devise a perturbation generator which confines the perturbations within the sensitive regions that are most critical for website identification of multiple surrogate models. Injecting perturbations in these sensitive regions can effectively deceive the closed-box WF attack model and reduce the bandwidth overhead, which however is rarely considered in the literature.
- We propose an RL-based framework to learn the optimal perturbation position and magnitude under the guidance of reward signal from a glass-box query model. To enhance the robustness against adversarial training, we introduce a random disturbance factor into the reward, which could induce diverse perturbations even for the same website.
- We develop a perturbation aggregation method to find a common perturbation paradigm for different traces of the same website, which enables us to conduct real-time dummy packet injection. By aggregating various perturbations, this paradigm further enhances the defense performance against closed-box WF attack models.
- We conduct extensive experiments using two widely used datasets to evaluate the performance. Experimental results show that RLpatch achieves a lower attack success rate, lower bandwidth overhead and stronger robustness against adversarial training compared to the state-of-the-art methods.

The remainder of this paper is organized as follows. Section II briefly reviews the related work. Section III describes the preliminaries and challenges. Our motivation is introduced in Section IV. Section V gives the details of our proposed adversarial WF defense method. Section VI presents some comparative experiments to demonstrate the

effectiveness and robustness of our method. Section VII discusses the deployment issues and limitations of our defense. The conclusion is drawn in Section VIII.

II. RELATED WORK

A. WF Attacks

WF attacks in the early stage leveraged expert knowledge to analyze the unique statistical features of different target websites, and employed machine learning classifiers such as KNN [11], random forest [12], and improved support vector machine [13] to identify target websites. Hermann et al. [14] were the first to propose the concept of WF attacks, which utilized the frequency distribution of packet lengths to classify websites. However, due to the fixed size (512 bytes) of Tor cells, the recognition accuracy was still far from satisfactory. Wang et al. [11] combined various features including packet ordering, the number of incoming/outgoing packets, and burst count into a multidimensional feature vector, and utilized a KNN classifier to identify the network traffic. To make a step forward, Hayes and Danezis [12] analyzed the importance of different statistical features using random forests, significantly reducing the number of input features. Penchenko et al. [13] proposed CUMUL, which extracted a fixed number of cumulative packet length features from traffic traces and identified website using an RBF kernel-based SVM.

Recent WF attacks usually achieved high recognition accuracy by utilizing deep learning methods to learn the sequence features of target websites automatically with side-channel information. Rimmer et al. [15] employed three types of DNN architectures, namely SDAE, CNN, and LSTM, for website fingerprinting. Afterwards, Sirinam et al. [16] developed a more complex CNN based WF model, which further improved the attack accuracy. They also examined the effectiveness of the proposed model against WF defense. Bhat et al. [17] designed a complex Var-CNN architecture to extract the temporal information of traffic traces as meta-features, which were fused with sequence features. Shen et al. [18] proposed a traffic aggregation matrix (TAM) based WF method by combining temporal features with sequential features.

Moreover, several recent studies have extended traditional WF attacks to specific scenarios. Yuan et al. [19] introduced class incremental learning into WF attacks, enabling the attack model to continuously expand its monitoring scope by learning new website classes. Zou et al. [20] proposed Deep Quadruplet Fingerprinting (DQF), which combines metric learning and meta-learning techniques to effectively tackle the few-shot learning problem in WF attacks. Meng et al. [21] have begun to address the challenges of the multi-tab WF attacks.

B. WF Defenses

To defeat WF attackers, defenders employ strategies such as adding dummy packets and delaying packets to alter the traffic patterns generated during website visits, thus confusing the attacker's classifier. We divide WF defense methods into three categories: obfuscation defenses, regularization defenses and adversarial defenses.

TABLE I
OVERVIEW OF NOTABLE WF DEFENSE

Ref.	C1		C2	C3
	Practicality	Efficiency	Effectiveness	Robustness
[6], [7]	✓	×	×	×
[22]–[25]	×	×	✓	×
[26], [27]	×	✓	×	×
[28]–[30]	✓	✓	×	×
[31], [32]	✓	✓	×	✓
Ours	✓	✓	✓	✓

(1) *Obfuscation Defenses*: The obfuscation defenses attempted to confuse attackers' classifiers by introducing uncertainty. By injecting random number of dummy packets or padding random delay between packets, they could ensure that the traffic patterns generated by different users even when visiting the same website were different. The representative method was WTF-PAD, an adaptive defense system proposed by Juarez et al. [6], which filled in idle segments of traffic with packets to disrupt the burst characteristics of traffic traces. Gong and Wang [7] observed that the initial portion of a traffic trace was critical for WF classification, so they inserted random packets into the heads of traffic traces to increase the difficulty for attackers to identify the traffic. However, the research in [16] has shown that deep learning based WF methods exhibit a certain degree of immunity against the obfuscation defense.

(2) *Regularization Defenses*: The basic idea of regularization defenses were designed to match the traffic pattern of a target website with that of another pre-defined website. Some regularization defenses, such as CS-BUFLO [22] and Tamaraw [23] aimed to hide fine-grained traffic features by shaping all website traffic into a unified pattern. Other regularization defenses like W-T [24] and Palette [25] clustered similar traffic into groups and generated similar traffic patterns within the clusters, making it difficult for attackers to distinguish between websites in the same cluster. However, regularization defenses could result in extremely high time and bandwidth overhead, making them impractical for deployment in Tor.

(3) *Adversarial defenses*: Adversarial defenses represented a new direction in WF defense design, exploiting the inherent vulnerabilities of deep neural networks. By injecting small adversarial perturbations into the traffic, these defenses aimed to mislead WF models, often with minimal bandwidth overhead compared to previous WF defense methods.

The earliest adversarial approach in WF defense scenario, named Mockingbird, was proposed in [26]. It generated perturbations by minimizing the distance between the traffic trace of the original website and that of a target website. However, this method lacked adaptability for real-time traffic, as it required the complete traffic trace until the session ended to generate the perturbations. To address this issue, Nasr et al. introduced Blind [27], which generated real-time perturbations by determining whether to inject perturbations based on the current traffic trace and injecting appropriate perturbations while meeting network traffic constraints. However, Blind also lacked feasibility during real-time injection, as it needed to

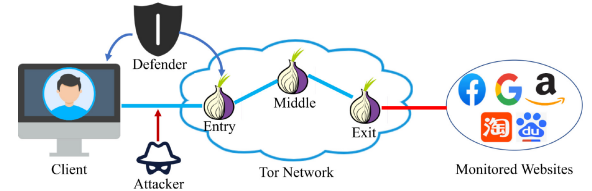


Fig. 1. The threat model of WF attack and defense.

evaluate each coming packet, leading to significant computational overhead and time delays.

To enable flexible real-time perturbation injection, Dolos [28] proposed generating a universal perturbation for each website. This perturbation is pre-calculated offline and applied online to indicate where and how many dummy packets should be injected. However, a limitation of Dolos was that it required glass-box access to the attacker's model architecture and parameters, which may not be a realistic assumption in practical scenarios. To address the glass-box assumption, AWA [29] introduced a transfer-based adversarial approach. In this approach, a glass-box model was used as surrogate model to generate universal perturbations, which are then transferred to unknown WF attack models for performance evaluation. Specifically, AWA randomly selected two websites and trained a GAN network to minimize the distributional differences between the two websites. Due to poor defense accuracy of transfer-based approach, Minipatch [30] introduced a query-based approach to generate adversarial perturbations, assuming that the defender can query the output of the attackers' WF model for given inputs. It utilized a simulated annealing algorithm and queried to find appropriate perturbation positions and sizes, offering better bandwidth efficiency and defense performance than previous methods. However, Minipatch was ineffective against WF attacks involving retraining the classifier. Acup3 [31] and Kimerapad [32] were proposed to attempt to solve this problem, and achieved some success, but they could not strike a good balance between bandwidth overhead and defense performance.

In summary, by comparing the advantages and disadvantages of previous works, we found that these methods are either limited by high computational and bandwidth overhead or by the adaptability in real-world deployment. Our approach aims to achieve good defense performance with low bandwidth overhead while meeting the requirements of real-time injection, closed-box effectiveness, and robustness against adversarial learning in real-world scenarios. We compare the key properties of WF defense methods against the challenges mentioned in Section III-D to our proposed method in Table I.

III. PRELIMINARIES AND CHALLENGES

A. Threat Model

Fig. 1 illustrates a typical WF attack and defense scenario. Users access a website using the Tor network to conceal their browsing activities. We follow the same assumption as in prior works [14], [15], [16], where the attackers are local eavesdroppers positioned between the user and the Tor

TABLE II
CAPABILITIES OF ATTACKERS AND DEFENDERS

Role	Capabilities
Attackers	- Passively monitor encrypted traffic (cannot modify or drop packets). - Collect traffic from monitored websites using their own Tor client. - Train WF models using packet directions only. - Can retrain WF models on perturbed traffic.
Defenders	- Know the set of websites the user wants to protect. - Collect traffic traces in advance and train defense models. - Use public white-box WF models as potential target models to defeat. - Can monitor the current traffic traces but cannot know the subsequent traffic traces. - Inject dummy packets in both directions without delaying real packets.

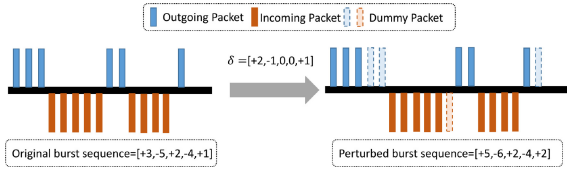


Fig. 2. An example of perturbation injection for burst sequence.

entry node. They can passively capture the encrypted traffic generated by the users' website browsing but cannot actively modify, discard, or decrypt the packets. Potential attackers might be network eavesdroppers, local network operators, or ISPs (Internet Service Providers). By analyzing side-channel information from the traffic traces, such as packet size, timestamp and direction, the attackers can pry into which website the users are visiting.

Before launching the WF attack, the attackers operate their own Tor client to collect the traffic of each website in the monitored set. Then the attackers train a classifier based on their collected traffic. Once the classifier is well-trained, the attackers monitor and collect users' traces and use the classifier to identify them as belonging to the websites in the monitored set.

On the defense side, we assume the defenders know the set of sensitive websites the users wish to protect (aligned with the attackers' monitored set), and can collect traffic traces accordingly. The defenders have access to public glass-box WF models for generating perturbation paradigms, and can inject dummy packets into live traffic without delaying original packets. They have the prior knowledge of which website the users are accessing but they can only observe current traffic bursts and not future patterns.

To ensure a rigorous evaluation, we adopt the widely used single-tab and closed-world setting where users only visit a website in the monitored set at a time. As this assumption benefits the attacker, it sets a higher bar for the defense. If our method performs well under such strict conditions, it is expected to be more robust in real-world scenarios with multi-tab and open-world setting where attacks are inherently more difficult. In this scenario, the capabilities of the attacker and defender are summarized in Table II.

B. Traffic Representation

In the Tor network, all packets are padded to a fixed size for encryption and transmission. Accordingly, the original traffic trace is denoted as a direction sequence $\mathbf{p} = [p_1, p_2, \dots, p_n]$, where $p_i = 1$ indicates an outgoing packet and $p_i = -1$ indicates an incoming packet. The defenders may inject dummy packet both in the client side and in the Tor entry side to break the sequential patterns. However, injecting opposite-direction dummy packets, *e.g.*, a dummy incoming packet between two outgoing packets or a dummy outgoing packet between two incoming packets, may interrupt the transmission process and cause an additional time delay, thus degrading the quality of users' browsing experience. To avoid this problem, defenders prefer to inject same-direction dummy packets at the tail of a traffic burst (*i.e.*, continuous packets with the same direction) [26], [30], [33]. The original traffic trace can be represented as a burst sequence $\mathbf{x} = [b_1, b_2, \dots, b_l]$, where $|b_i|$ indicates the packets number and the sign of b_i indicates the packet direction of the i -th traffic burst. Here, $b_i > 0$ represents outgoing burst and $b_i < 0$ represents incoming burst. Fig. 2 shows an example of packet injection for burst sequence. The injected dummy packets have the same direction as the burst, ensuring that the communication between users and the website is not disrupted.

C. Adversarial Perturbation Based WF Defense

Existing adversarial WF defenses are all inspired by adversarial learning, where adding small, well-designed perturbations into the correctly classified input can mislead DNN-based classifier. Given a clean burst sequence $\mathbf{x}$, its corresponding ground-truth label, denoted as y , belongs to the monitored set $Y = \{1, 2, \dots, K\}$. Let f^b denote the closed-box WF attack model, both the structure and parameters of which are unknown to the defenders. The aim of WF attackers is to map the burst sequence $\mathbf{x}$ to its correct label y , *i.e.*, $f_b : \mathbf{x} \rightarrow y$. Conversely, the defenders aim to generate a perturbed sequence $\mathbf{x}^{adv}$ to deceive the attack model f^b , *i.e.*, $f^b(\mathbf{x}^{adv}) \neq y$, where $\mathbf{x}^{adv}$ is defined as:

$$\mathbf{x}^{adv} = \mathbf{x} + \mathbf{q} \odot \boldsymbol{\delta}, \quad (1)$$

where $\mathbf{q} = [q_1, \dots, q_l]$ is a binary mask sequence with the same length as $\mathbf{x}$, indicating the actual injection position of the perturbations. If a perturbation is added behind the i -th burst, $q_i = 1$. Otherwise, $q_i = 0$. $\boldsymbol{\delta} = [\delta_1, \dots, \delta_l]$ determines the magnitude of perturbations, where each element δ_i specifies the number of dummy packets injected behind the i -th burst. $\mathbf{q} \odot \boldsymbol{\delta}$ represents the perturbation sequence inserted into $\mathbf{x}$, where $\odot$ denotes the Hadamard product.

By jointly optimizing $\mathbf{q}$ and $\boldsymbol{\delta}$, we aim to maximize the misclassification probability of the attack model f^b while keeping a low bandwidth overhead. Formally, the optimization problem can be formulated as:

$$\mathbf{x}^* = \arg \max_{\|\mathbf{q} \odot \boldsymbol{\delta}\|_1 < \beta} \mathbb{E}_{(\mathbf{x}, y) \in D} \left(f^b(\mathbf{x}^{adv}), y \right), \quad (2)$$

where the constraint $\|\mathbf{q} \odot \boldsymbol{\delta}\|_1 < \beta$ ensures that the total amount of injected dummy packets does not exceed a

predefined budget β . D is the set of traffic traces that are protected from WF attack. $L(\cdot)$ measures the deviation between the predicted label of $f^b(x^{adv})$ and the actual label y .

D. Challenges

To solve the above problem, several realistic challenges must be addressed:

- **Zero Delay and Low Overhead (C1):** During the WF defense, padding dummy packets may increase the bandwidth overhead, while delaying the transmission packets may increase the time overhead. In the Tor network, since users still have a high expectation for smooth network experiences, bandwidth and time overheads are crucial considerations in the design of WF defense mechanisms. However, most existing defense methods, such as [22] and [24], insert perturbation throughout the traffic traces, which may cause intolerable delays and significant bandwidth overhead. Thereby, a practical WF defense solution is expected to achieve zero delay and keep a low bandwidth overhead, that is, β in Eq. (2) is expected to be small.
- **Effectiveness under Closed-Box Setting (C2):** In a practical scenario, the attackers may employ diverse tactics for conducting WF attacks, *i.e.*, the structure and parameters of the attack model f^b in Eq. (2), are unknown to the defenders. It is challenging to generate an effective adversarial perturbations against such a closed-box attack model. Existing defense researches, such as [26] and [31], directly transfer the adversarial perturbation generated from a given glass-box model to a closed-box attack model. However, due to the discrepancies in attack models, the transferability of adversarial perturbations across different models is often limited, resulting in a poor defense performance under closed-box setting.
- **Robustness against Adversarial Training (C3):** The WF defense methods are mostly implemented through pluggable transports (PT) [34] to adapt to Tor network. Attackers can collect defense traffic traces using the same defense settings as users and retrain their attack models to penetrate the established WF defenses. Yet, current methods such as [27] and [30], are not robust enough to adversarial training, because they lack a user isolation policy to ensure that users visiting the same website are assigned different perturbations.

In summary, the above challenges necessitate that the WF defense method should be low-overhead, effective and robust.

IV. MOTIVATION

Existing studies typically assume that each region of a traffic trace has an equal impact on the decision-making of WF attack model. As a result, the perturbations have to act on the entire traffic trace, greatly increasing the bandwidth overhead. However, we have an insight that only a few sensitive regions

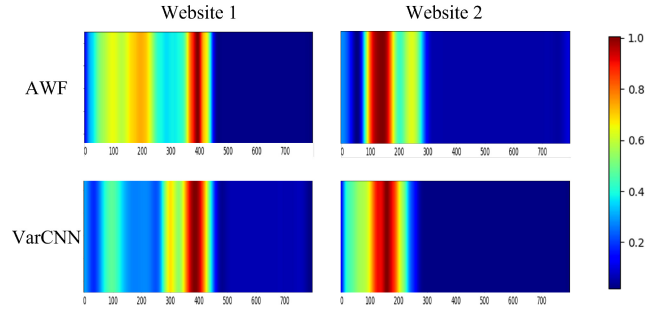


Fig. 3. Heatmaps of two burst sequences for different WF attack models.

can actually affect the decision-making of WF attacks. As shown in Fig. 3, we employ the Grad-CAM method [35] to generate a heatmap that highlights the contribution of different regions of a burst sequence on the decision-making of WF attack models. The regions with warm colors, *i.e.*, red or yellow, have a great impact on the WF model's decision, while the regions with cool colors, *i.e.*, blue, have a small impact. Fig. 3 reveals that the WF attacks may focus on several critical regions of the burst sequence, which play a more important role in WF than other regions. These regions are usually associated with the loading of website styles (*i.e.*, CSS, JavaScript, fonts, $\cdot \cdot \cdot$), the response of specific user requests, and the transmission of specific resources, which often represent the unique fingerprint of a specific website. Injecting perturbations into these regions can erase the distinctive fingerprint of the traffic trace and mislead the WF attack model. That is, these regions may be highly sensitive to perturbations, where even a small perturbation can lead to a significant change of the WF model's decision. These finding inspires us to apply perturbations only on a few sensitive regions, rather than on the entire trace, which could significantly reduce the number of dummy packets as well as the bandwidth overhead, thus addressing C1.

From Fig. 3, we also find that different websites may have different sensitive regions, but different WF attack models may share some common sensitive regions for the same website. These common sensitive regions are very likely to be crucial for various WF attack models, even for the unknown closed-box WF attack models. This findings inspire us to injecting perturbations into the common sensitive regions for enhancing the defense performance against the closed-box WF attack models, thus addressing C2.

In Fig. 3, if we not only erase the distinctive fingerprint of *website1* in the region between the 360th and 420th burst, but also add perturbations in the region between the 100th and 200th burst, we can make the perturbed sequence of *website1* look like *website2*, so as to deceive the potential WF attackers. Here, *website2* is the imitation target. In practice, to enhance the robustness of defense model, different users may randomly choose their own imitation targets, which are usually not available to the attackers. In this way, even when the attackers use the same defense model to collect the perturbation samples for adversarial training, they can hardly get the same adversarial samples as the defenders, thus addressing C3.

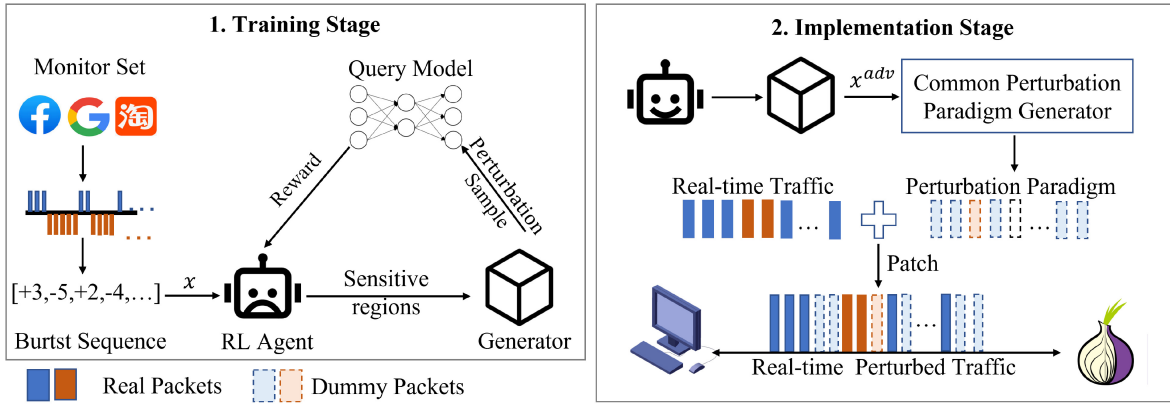


Fig. 4. An overview of RLpatch.

V. METHODOLOGY

A. System Overview

In light of the above insights, we develop a WF defense framework named RLpatch which aims to achieve good real-time defense performance within sensitive regions under closed-box setting. The workflow of RLpatch is illustrated in Fig. 4, which can be divided into two stages: training stage and implementation stage. In the training stage, the collected traffic traces of target websites in the monitored set are represented as burst sequences, which are used for defense model training. The RL agent is designed for locating the sensitive regions where the perturbations should be applied on, and the generator is responsible for determining how to inject the perturbation within the given sensitive region. Here, we can choose a strongest glass-box WF attack model we know as the query model to defeat. The query model assesses the effectiveness of the crafted perturbations and drives the RL agent to improve the defense performance. During the training, the RL agent can learn an optimal perturbation policy through numerous rounds of interactions with the query model.

In the implementation stage, we can generate an adversarial perturbation for each samples of a website using the well-trained RL agent. Accordingly, we can get different perturbation sequences for the same website. The common perturbation paradigm generator is designed for producing a uniform perturbation paradigm for a website. Once the common perturbation paradigms are generated, the real-time injection phase no longer requires the involvement of the RL agent, as these common perturbation paradigms can serve as a fixed configuration deployed to the PT proxy of both the user and Tor entry node. In this way, the PT proxy can dynamically switch the perturbation paradigm just according to the website that users are visiting. Every time the user or the Tor entry node has a packet burst to send, the PT proxy can directly determine the number of dummy packets for injecting according to the selected perturbation paradigm and the current burst position. Afterwards, the dummy packets are injected to the tail of the current packet burst and sent together with the real packets until the website is loaded, thus achieving zero-delay perturbation injection.

B. Training Stage

1) *Design of Perturbations Generator*: For a given sensitive region, the generator is responsible for crafting an adversarial perturbation. Suppose a sensitive region starts from position c with a length of k , i.e., the c -th to $(c + k - 1)$ -th burst of the raw sequence x . Accordingly, the binary mask sequence for such sensitive region is denoted by

$$q^c[i] = \begin{cases} 1 & \text{if } c \leq i < c + k \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

According to Eq. (1), the perturbed sequence is $x + q^c \odot \delta$. The remaining problem is to determine the perturbation magnitudes δ within the sensitive region. To achieve this, an intuitive idea is to explore every possible perturbation at each position of the sensitive region. However, the vast search space makes it almost impracticable to apply in practice.

To address this problem, we employ the gradient feedback from a glass-box surrogate WF attack model, whose structure and parameters are accessible, to guide the direction of perturbation generation, rather than blindly trying all possible perturbation magnitudes. Because the gradient of surrogate WF attack models actually represents the direction that can increase the deviation between the predicted result and the ground-truth result. Adding perturbations along the gradient direction may induce great errors in WF attack model's predicted results, thus deceiving the WF attackers. Meanwhile, to enhance the generalization ability of the defense model against C2, we can synthesize the gradients from n surrogate WF attack models which are different from the query model to find a common direction that can effectively deceive a broader range of attack models. Let f_i^s denote the i -th surrogate WF model. The normalized gradient is defined as

$$g = \frac{\sum_{i=1}^n \rho_i \nabla_{\delta} L(f_i^s(x + q^c \odot \delta), y)}{\|\sum_{i=1}^n \rho_i \nabla_{\delta} L(f_i^s(x + q^c \odot \delta), y)\|_1}, \quad (4)$$

where ρ_i denotes the weight of the i -th surrogate model and $\nabla_{\delta} L(\cdot)$ denotes the gradient of deviation function $L(\cdot)$ with respect to the perturbation magnitude δ . As the surrogate model is glass-box, the value of $\nabla_{\delta} L(\cdot)$ is calculable.

In practice, we can obtain the optimal perturbation magnitude through an iterative way. Here, we leverage

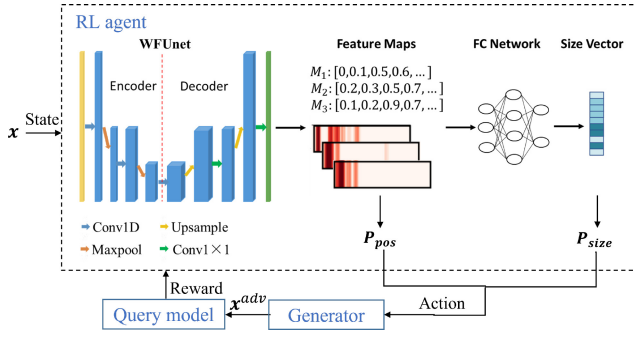


Fig. 5. Design of RL agent.

the Momentum Iterative Fast Gradient Sign Method (MIFGSM) [36] to iteratively adjust the perturbation. Let t denote the iteration round. Initially, both the perturbation δ_0 and gradient g_0 are set to 0. During the iteration, we adopt the accumulated gradient to smooth the fluctuation of gradient, which is updated as follows:

$$\tilde{g}_t = \mu \cdot \tilde{g}_{t-1} + g_t, \quad (5)$$

where g_t is the feedback gradient at the t -th interaction with the surrogate models, which can be calculated by Eq. (4), and μ represents the attenuation coefficient, which is used to adjust the impact of the previous gradients on the current gradient.

Along the gradient direction, we accordingly adjust the perturbation by:

$$\delta_t = \delta_{t-1} + \epsilon \cdot \text{sign}(\tilde{g}_t), \quad (6)$$

where $\text{sign}(\cdot)$ represents the gradient direction, and ϵ is the perturbation step size. For a given sensitive region, the perturbation magnitude δ is determined by the step size ϵ . In particular, a large ϵ may induce excessive dummy packet injections, but a small ϵ leads to too many iterations.

Notably, the direction of the generated perturbation may be opposite from the direction of the raw burst sequence. Such inconsistency may interrupt the actual data transmission and introduce extra delay as aforementioned in Section III-B. Thereby, we should further calibrate the perturbation by dropping the opposite-direction perturbation as follows:

$$\delta_t[j] = \begin{cases} 0 & \text{if } \delta_t[j] < 0 \text{ and } x[j] > 0, \\ 0 & \text{if } \delta_t[j] > 0 \text{ and } x[j] < 0. \end{cases} \quad (7)$$

The perturbed sequence is then given by:

$$x_t^{adv} = x + q^c \odot \delta_t. \quad (8)$$

Afterwards, we have to input x_t^{adv} into the surrogate models to get g_{t+1} for a next iteration. The above iteration process would be continued until the gradient g_t remains relatively unchanged or the maximum number of iterations is reached, when the generated perturbation is considered optimal.

The sensitive regions c and perturbation size ϵ determine where to add perturbation and how many dummy packets to inject, respectively. For a specific combination of c and ϵ , we can generate an x^{adv} by iteratively interacting with the surrogate models. Thereby, the remaining crucial problem is to figure out optimal c and ϵ , which would be discussed below.

2) *Design of RL Agent:* Since different positions may require different perturbations, we have to optimize the perturbation position and magnitude jointly. To achieve this, we employ RL to learn the optimal perturbation policy under the guidance of reward signal from the strongest glass-box query model f^q . For a burst sequence x , the RL agent gives an action of perturbation position c and size ϵ , and the generator then produces an adversarial perturbation sequence x^{adv} . By interacting with the query model, we can acquire the corresponding reward, i.e., the defense performance. Afterwards, we can fine tune the parameters of RL agent to steer the selection of c and ϵ towards the direction of maximizing the reward.

The related state, action and reward are defined as follows.

State: The state s represents the burst sequence x inputted to the RL agent.

Action: The action $a = (c, \epsilon)$ is defined as the perturbation position and step size. The RL agent has to learn two policies: position policy and size policy.

The position policy decides where to add the perturbation in the burst sequence can mislead the closed-box WF model. As depicted in Fig. 5, we employ an autoencoder-like structure [37] named WFUnet, whose shape is like a ‘U’, to learn the position policy. WFUnet utilizes an encoder composed of convolution layers and max-pooling layers to extract the high-level features of input burst sequences and a decoder composed of upsampling layers and convolution layers to output the feature map. Each element in the feature map measures the contribution of corresponding burst on decision-making of WF attack. The length of feature map is same with the input burst sequence. We employ the decoder of WFUnet to generate a separate feature map M_i for each surrogate model f_i^s , providing different insights into the sensitivity of each position. The overlapped sensitive regions of all feature maps represent the distinctive fingerprint of a specific website, which could be also used for WF recognition by other closed-box models. By aggregating all feature maps, we can calculate the mean sensitivity of each position as follows:

$$P_{pos} = \frac{1}{n} \sum_{i=1}^n \text{softmax}(M_i), \quad (9)$$

where $\text{softmax}(\cdot)$ normalize the feature map to $[0,1]$. A greater $P_{pos}[j]$ indicates that adding perturbation in j -th position of burst sequence x is more likely to mislead the WF attack model. Therefore, such position should have a higher likelihood to be selected as the start position of sensitive region. We can utilize a categorical distribution function to model the position policy, as it is well-suited for representing the agent’s probabilistic choices over a discrete set of positions. In this way, the start position c of sensitive regions can be sampled from $c \sim \text{Cat}(P_{pos})$. The length of a sensitive region depends on the number of continuous elements in the feature map greater than a specific threshold. As different surrogate models have different impacts on the gradient, we need to pay more attention to the surrogate models that exhibit higher sensitivity to the burst sequence. The weight ρ_i defined in Eq. (4) is set

to $\frac{\overline{M_i}}{\sum_{j=1}^n \overline{M_j}}$, where $\overline{M_i}$ is the mean value of all elements in the feature map f_i^s .

With such feature map, we further employ a fully connected network (**FC**) to learn a perturbation size policy, which establishes a mapping relationship between the perturbation positions and the perturbation size. Specifically, the perturbation step size ϵ is selected from a series of discrete values. The probability distribution of selecting different step sizes is represented as:

$$P_{size} = \text{softmax}(\text{FC}(P_{pos})). \quad (10)$$

Similarly, we employ the categorical distribution function to model the magnitude policy. The step size ϵ can be sampled from $\epsilon \sim \text{Cat}(P_{size})$.

Reward: The agent is trained to identify sensitive regions within the input burst sequence x , where perturbations can be generated and applied to maximize the likelihood of causing misclassification of the query model.

To achieve this, we define the reward R based on the misclassification probability of the perturbation. Due to the existence of **C3**, the attackers and defenders may use the same reward function to generate similar perturbation samples and penetrate the defense via adversarial training. In particular, if directly applying the output of query model, *i.e.*, $L(f^q(x^{adv}, y))$, as the reward, when the attackers query the same WF model as the defenders, they can easily penetrate the defense through adversarial training.

To address the problem, we should introduce randomness into the reward function to create diverse perturbations for the same burst sequence. Specifically, we multiply the output of query model by a disturbance factor, which is randomly initialized by the defenders. The reward is defined as:

$$R = \sum_{y_i \in Y, y_i \neq y} \eta_i \cdot f^q(x^{adv}, y_i), \quad (11)$$

where $\eta_i \in (0, 1]$ is the disturbance factor, and $f^q(x^{adv}, y_i)$ denotes the output probability that the query model misclassified the perturbed sequence x^{adv} with a ground-truth label y as the label y_i . Each defender may set its own disturbance factor, which is usually unknown to the attackers. Varying the disturbance factor may change the misclassification results as well as the direction of perturbation generation. In this way, the defenders can generate diverse perturbations for the same raw sequence, thus addressing **C3**.

In some specific scenarios, the defenders may specify a target website label $\hat{y}$ ($\hat{y} \neq y$), that is, make the attack model misclassify the sequence x to a certain target label $\hat{y}$. In this case, the defenders can generate distinct perturbations by selecting different $\hat{y}$, which is generally not accessible to the attackers, ensuring the robustness against the attackers' adversarial training. Thereby, the reward can be defined as $R = f^q(x^{adv}, \hat{y})$ for the targeted scenario.

3) *Policy Update:* During model training, for each raw traffic sequence x , the generator crafts an adversarial perturbation x^{adv} based on the action $a = (c, \epsilon)$ of the RL agent. By interacting with the query model, we can evaluate the effectiveness of such perturbation using the reward defined in Eq. (11).

Afterwards, the RL agent would refine its perturbation policy denoted by π_θ where θ is the parameter set of RL agent. The aim of RL agent is to learn a set of optimal parameters θ^* to maximize the expected reward, which is formulated as:

$$\theta^* = \arg \max_{\theta} J(\theta), \quad (12)$$

where $J(\theta) = E_{\pi_\theta}[R(s, a)]$ represents the expected reward based on state s and action a under the current policy π_θ . To maximize $J(\theta)$, we have to figure out the policy gradient $\nabla_\theta J(\theta)$, which provides the direction for parameters updating. Here, we employ the Monte Carlo Method [38] to approximate $\nabla_\theta J(\theta)$ by averaging the rewards over different actions for the current policy π_θ . Specifically, we sample N actions $\{a_1, a_2, \dots, a_N\}$ from the policy π_θ and obtain the corresponding rewards $\{R_1, R_2, \dots, R_N\}$ through the query model, thus forming a series of state-action-reward tuples $(s, a_1, R_1), (s, a_2, R_2), \dots, (s, a_N, R_N)$. The policy gradient is estimated by:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log \pi_\theta(a_i | s) R_i. \quad (13)$$

where $\nabla_\theta \log \pi_\theta(a_i | s)$ is calculated using the back propagation algorithm since the policy π_θ is estimated by the WFUnet and FC network.

With such policy gradient, the agent can update its policy parameters θ using the gradient ascent method. θ is updated by $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J(\theta)$, where α denotes the learning rate. A large gradient $\nabla_\theta J(\theta)$ may result in a large reward R , indicating that such perturbation would mislead the WF attack model towards the direction away from the ground-truth label. In this way, updating the parameters along the policy gradient direction would increase the reward during the training process. The above training process would be continued until the gradient $\nabla_\theta J(\theta)$ remains relatively unchanged, when the perturbation policy is considered to be optimal. The complete process of policy update is presented in Algorithm 1.

C. Implementation Stage

The well-trained RL agent cannot be directly deployed in practice. Because the decision making of RL agent requires a prior knowledge of the complete traffic sequence x , which however cannot be obtained in advance especially for an ongoing session where the proxy and the client are interacting with each other. In other words, we can know which website the client is visiting but cannot know the complete traffic trace for a real-time defense scenario. Actually, even visiting the same website may induce different samples of traffic traces, due to the diversity of user behavior and the dynamic of network condition. In particular, with such well-trained RL agent, we craft an adversarial perturbation sequence $v = x^{adv} - x$ for each individual sample x in the training set. We can accordingly acquire a series of various perturbations for the same website w denote by $V^w = \{v_1^w, v_2^w, \dots\}$. The pre-generated perturbations designed for one trace may not work well on other traces of the same website. A straightforward idea is to overlay all perturbation sequences from website w .

Algorithm 1 Crafting Adversarial Perturbation

Input: burst sequence x , truth label y , query model f^q , surrogate models f_i^s ($i = 1, 2, \dots, n$), maximum number of query Z , number of samples N , learning rate α

- 1: **Initialization policy parameter** θ ;
- 2: **for** each burst sequence x **do**
- 3: **for** $z = 1, 2, \dots, Z$ **do**
- 4: Calculate feature maps M ;
- 5: Calculate position policy and perturbation size policy using Eq. (9), (10);
- 6: **for** $i = 1, 2, \dots, N$ **do**
- 7: Sample actions $a = (c, \epsilon)$;
- 8: Calculate binary mask q^c by Eq. (3)
- 9: Calculate x^{adv} by Eq. (5), (6), (7), (8);
- 10: Calculate reward R_i ;
- 11: **end for**
- 12: Calculate policy gradient $\nabla_{\theta} J(\theta)$ by Eq. (13);
- 13: Update the agent parameter $\theta = \theta + \alpha \cdot \nabla_{\theta} J(\theta)$;
- 14: if $(f^q(x^{adv}) \neq y)$ then break;
- 15: **end for**
- 16: **end for**

This method may achieve a high defense performance, as it covers a wide range of vulnerabilities within the model. However, it may also result in a very high bandwidth overhead.

To address this issue, in the implementation stage, we attempt to find a common perturbation paradigm for a website which could be effective across different traffic traces for the same website, rather than generate a perturbation for each single traffic trace. Our basic idea is to aggregate different perturbations for the same website based on their positional frequency. The procedure of our method consists of three steps:

Frequency calculating: Intuitively, if the well-trained RL agent decides to pad perturbation at the same position for most of the samples in V^w , such position would be the vulnerable point for website w . In light of this, we employ a frequency metric Fre_i^w to quantify how often a perturbation is added at a specific position i , which is defined as:

$$Fre_i^w = \sum_{j=1}^{|V^w|} I(v_j^w[i]), \quad (14)$$

where $I(\cdot)$ is an indicator function. If $v_j^w[i] \neq 0$, $I(\cdot) = 1$, otherwise $I(\cdot) = 0$. A larger Fre_i^w indicates that the position i is more likely to induce successful perturbation.

Perturbation sort: We rank all positions according to its frequency Fre^w in a descending order and only retain the top H positions with the highest frequency, which form a list B^w . These positions in B^w represent the common sensitive regions for all samples of website w , which are most commonly associated with successful perturbation. Padding perturbation only at these positions is more likely to deceive unknown WF attack model which may also concentrate on

Algorithm 2 Perturbation Paradigm Generation

Input: Perturbation sequence set V^w , surrogate models f_i^s ($i = 1, 2, \dots, n$), accuracy threshold λ

Output: The Perturbation Paradigm U^w of website w

- 1: Initialize U^w by 0;
- 2: Calculate Fre_i^w for each position i by Eq. (14)
- 3: Rank all positions in descending order of Fre_i^w and select the top H positions to form a list B^w
- 4: Calculate the average perturbation size D^w
- 5: Initialize the accuracy $E = 1$
- 6: **for** each position $i \in B^w$ **do**
- 7: Update $U^w[i] = D^w[i]$
- 8: Evaluate the accuracy E_b of current U^w in all the surrogate models
- 9: **if** $E - E_b > \lambda$ **then**
- 10: $E = E_b$;
- 11: **else**
- 12: $U^w[i] = 0$;
- 13: **end if**
- 14: **end for**

these characteristic positions, thus enhancing the performance against C2.

Perturbation aggregation: We can aggregate all perturbations of all samples in v^w as follows:

$$D^w[i] = \begin{cases} \frac{\sum_{j=1}^{|V^w|} v_j^w[i]}{Fre_i^w}, & \text{if } i \in B^w, \\ 0, & \text{otherwise,} \end{cases} \quad (15)$$

which is actually the average perturbation at these top-ranking positions. Actually, injecting perturbation at all position in v^w may still cause a high bandwidth overhead. Thereby, we employ an iterative method to further sift out a common perturbation paradigm U^w . U^w is initialized by 0. In each iteration, we select a position i from the list B^w based on the frequency order and retrieve the corresponding perturbation size from D^w . Append $D^w[i]$ to U^w , i.e., $U^w[i] = D^w[i]$. Afterwards, we can evaluate the effectiveness of current U^w by padding the perturbation U^w into all samples in v^w and assessing their defense performance with the surrogate models. If the decrease of the attack success rate induced by the perturbation U^w exceeds a threshold λ , retain $U^w[i]$; otherwise, drop it. The main procedure of perturbation paradigm generation is described in Algorithm 2. The above process iterates over the list B^w .

In the real-world implementation, the defenders need to gather traffic traces of each monitored website, and offline generate the common perturbation paradigms using RLpatch. Afterwards, these perturbation paradigms are deployed on the PT proxies of both the client side and Tor entry node. The PT proxies can select corresponding the perturbation paradigms according to the website visited by the clients. As the perturbation paradigms give which positions and how many dummy packets to inject, the PT proxy add perturbation to the live Tor traffic in the corresponding burst position until the session ends.

VI. EXPERIMENT RESULTS AND ANALYSIS

In this section, we conduct extensive experiments to verify the effectiveness of RLpatch. For performance comparison, we compare RLpatch with state-of-the-art (SOTA) WF defense methods in terms of defense performance, bandwidth overhead and robustness. To make our work reproducible, we have made our code publicly available on Github.¹

A. Experimental Settings

WF Dataset: We select the public available Siranam dataset [16] and Rimmer dataset [15] for performance evaluation, as done in most previous studies. The Siranam dataset consists of directional sequences from 95 monitored websites, each of which contains 1000 instances, chosen from the top 100 Alexa-ranked websites. The Rimmer dataset includes 100 monitored websites, each with 2500 examples. Similar to [26], we preprocess the data by removing instances with fewer than 50 packets or starting with incoming packets. After filtering, 460 instances per website are retained in Siranam dataset, and 1000 instances per website in Rimmer dataset. Since the traffic traces collected by the attackers and defenders may differ, we divide the datasets into three parts: defenders' training set, attackers' training set, and testing set. Specifically, in the Siranam dataset, 200 instances per website are used to train the defenders' surrogate and query models, another 200 for the attackers' model, and the remaining 60 for testing. In the Rimmer dataset, 400 instances per website are used for both training sets, and 200 for testing.

Attack Models: To evaluate the effectiveness and other properties of RLpatch on different DNNs-based WF attack models, we adopt four representative DNN-based WF attack models, namely AWF [15], DF [16], VarCNN [17], and TF [39], which have been widely used in the existing studies of WF defense. Note that the defense should be effective to various models the attacker may adopt in real-world scenarios. Therefore, it is realistic for the defenders to choose multiple surrogate models to enhance the generalization capability of the perturbations.

Evaluation Metrics: We employ two metrics: attack success rate (ASR) and the bandwidth overhead (BWO) to evaluate the performance of defense methods. ASR refers to the classification accuracy of the perturbed sequence on the attacker's WF model. The decline in ASR before and after applying defense represents the strength of the defense. Overall, a lower ASR indicates stronger defense effectiveness. BWO is defined as the ratio of the additional bandwidth, caused by the injected dummy packets introduced by the defense, to the original bandwidth of the real packets. Furthermore, we use the average number of queries (ANQ) as the metric to assess the query counts required for our method to generate appropriate perturbations.

Baselines: To comprehensively evaluate the defense performance of RLpatch, we compare it with seven SOTA WF real-time defense methods. Specifically, we select two representative methods, *i.e.*, WTF-PAD [6] and Palette [25]

TABLE III
PERFORMANCE FOR DIFFERENT DEFENSE METHODS

(a) Siranam Dataset					
Defense	ASR(%)				BWO (%)
	DF	VarCNN	AWF	TF	
No Defense	96.3	96.7	91.7	94.1	0
WTF-PAD	89.4	91.2	61.2	92.1	61.0
Palette	20.3	22.8	16.2	25.5	84
AWA	22.1	19.5	15.2	21.2	29.2
KimeraPAD	18.8	20.9	12.9	22.1	27.5
Dolos	15.9	16.5	10.1	18.7	30.6
Acup3	13.7	17.6	8.5	16.7	21.4
Minipatch	7.9	13.2	6.2	18.0	15.1
RLpatch	2.7	7.4	1.8	7.5	12.5

(b) Rimmer Dataset					
Defense	ASR(%)				BWO (%)
	DF	VarCNN	AWF	TF	
No Defense	98.1	97.9	94.3	95.2	0
WTF-PAD	90.8	94.8	69.3	93.8	72.3
Palette	24.5	29.1	18.9	32.4	80
AWA	27.9	29.1	18.4	29.7	29.5
KimeraPAD	25.6	24.1	16.7	25.8	29.4
Dolos	21.1	22.8	15.3	23.6	30.1
Acup3	18.3	19.2	15.4	20.2	22.2
Minipatch	12.5	16.6	10.2	19.9	15.6
RLpatch	7.5	10.9	2.6	14.4	12.6

from the categories of obfuscation defenses and regularization defenses respectively. Additionally, we choose five adversarial defense methods. Among them, AWA [29] and Dolos [28] are pioneering works on adversarial WF defense, which use universal perturbations to meet the real-time requirements and reduce the BWO to an acceptable level. Minipatch [30] achieves a high defense performance under glass-box setting but struggles with unseen closed-box WF model and adversarial training. Acup3 [31] and KimeraPAD [32] focus on enhancing robustness against adversarial training. We use ASR and BWO to measure the defense capability under various practical scenarios.

To ensure a fair comparison, we conduct experiments using the same training and testing datasets and the same attackers' WF models. In the offline training phase, we generate perturbation paradigms using the defenders' training dataset for each website. In the testing phase, we apply the perturbations to each trace in the testing dataset using different defense methods. In this way, we can obtain a corresponding perturbed testing dataset for each defense method. Meanwhile, we use the attackers' training dataset to train different attack models. These models are used to evaluate the ASR and the BWO in each perturbed testing dataset.

B. Comparison With Baselines

Overall Performance under Glass-Box Setting. We compare the ASR and BWO of our method with other defense methods when the defenders have prior knowledge of the WF attack models frequently used by the attackers. This scenario represents the upper bound of defense performance. We generate perturbations on known glass-box surrogate models and evaluate the accuracy of the perturbed samples on the same attackers' target WF attack models. Specifically, we first use the defenders' training dataset to train the surrogate models and the query models. Afterwards, for each trace in the testing

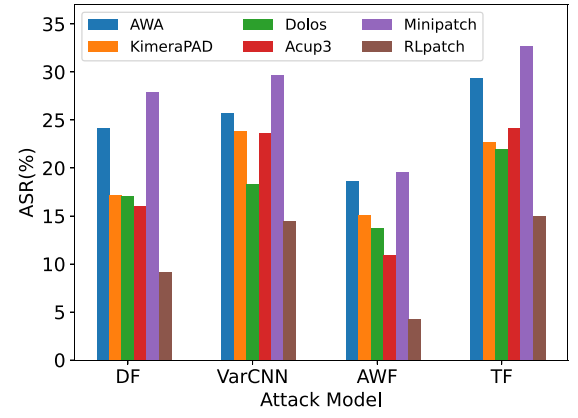
¹<https://github.com/chenxiailian/RLpatch-WF>

dataset, we add the corresponding perturbation generated by each defense method to the trace to obtain a new perturbed trace, thus generating the perturbed testing dataset specific to the method. Finally, we use the target WF attack models to test the ASR and the BWO of each perturbed testing dataset. The final results are shown in Table III.

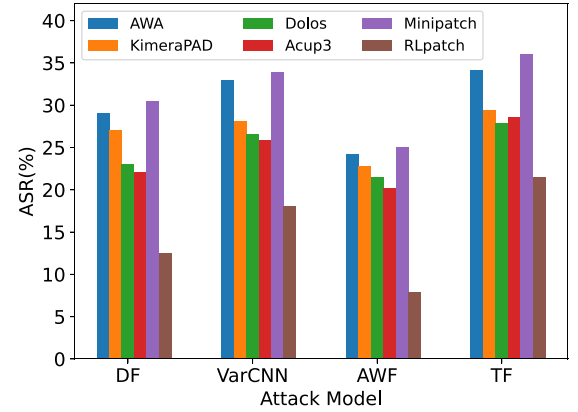
From the results, we can observe that WTF-PAD has the worst defense performance because its random perturbations fail to disrupt the attacker's classification patterns. Moreover, we can find that the camouflage defense Palette has an ASR similar to some adversarial defense methods, but because it needs to pad the traffic trace into a similar pattern, it has a higher BWO. Adversarial defense methods AWA, Dolos, Acup3, and KimeraPAD have similar ASR and BWO levels, indicating that their defense performances are comparable. We find that Minipatch even reduces the ASR on AWF to 6.2%, with a lower bandwidth overhead than the previous methods. By limiting the size and number of perturbations, Minipatch forces the perturbations to be added only in the positions that significantly affect the model's decision-making, thus reducing bandwidth overhead. Furthermore, we also find that our proposed RLpatch outperforms all the aforementioned methods, including Minipatch, in terms of ASR on the Siranam and Rimmer datasets. RLpatch strategically focuses its perturbation on sensitive regions that are consistently identified across various surrogate models, enabling RLpatch to generate effective perturbations with fewer dummy packet injections. RLpatch not only refines the perturbation in the sensitive region but aggregates different perturbation patterns for the same website, thus enhancing the effectiveness of the defense. Furthermore, by comparing the results on the Siranam and Rimmer dataset, we find that the defense performance on the Rimmer dataset is not as good as that on the Siranam dataset. This may be due to the larger size of the Rimmer dataset, which favors DL-based attack methods and enhances their classification capabilities.

Overall Performance under closed-box setting. To evaluate the effectiveness of well-trained defense models under a closed-box setting, we ensure that the surrogate and query models used for training are different from the attack model used for evaluation. In this setting, the defenders have no knowledge of the attackers' target model. This scenario represents the lower bound of defense effectiveness. We do not use WTF-PAD and Palette in this experiment because their mechanisms are model-agnostic. In this experiment, for RLpatch, we select two out of the four available WF attack models as the surrogate models, use another one as the query model, and treat the remaining one as the unseen closed-box model. Specifically, we generate perturbations in the glass-box setting using the surrogate model and query model. Once these perturbations are generated, we apply them in the testing dataset to create perturbed dataset and employ a target WF attack model that is unseen during training to test the ASR. The experiments are conducted on both the Siranam and Rimmer datasets for all adversarial defense methods.

Fig. 6 presents the ASR under closed-box settings. We find the ASRs in this experiment are all lower than those



(a) Siranam dataset



(b) Rimmer dataset

Fig. 6. The ASR against different closed-box WF models.

in previous experiment where the attack model is known to the defenders. The above result indicates that the disparity between the actual attackers' and the training models may induce the decline of defense performance. We also observe that Minipatch achieves the highest ASR among these defense methods. This result reveals that Minipatch can only generate effective perturbation against the query model. When the attackers' model differs from the query model, the defense performance drops sharply. In contrast, the defense methods including AWA, Acup3, KimeraPAD and Dolos rely on gradient information of surrogate model to generate perturbations. Since this gradient direction may exhibit some similarities across different WF models, these methods perform slightly better against unseen models than Minipatch through transferability. Compared to these approaches, RLpatch utilizes the weighted gradients from multiple surrogate models to generate perturbations in the identified sensitive regions. This shared perturbation direction enhances the generalization capability of the perturbations to deceive unseen models, thereby achieving the lowest ASR in the closed-box WF models.

Combined with the previous results, we can conclude that our defense method effectively improves the performance under both glass-box and closed-box settings, demonstrating its effectiveness and generalization capability across different attack models.

TABLE IV
PERFORMANCE AGAINST ADVERSARIAL TRAINING

(a) Untargeted scenario						
No. of configuration	ASR					BWO
	1	2	3	4	5	
KimeraPAD	65.1	69.5	72.1	76.1	80.5	28.2
Minipatch	60.4	96.1	98.5	99.3	99.5	15.8
RLpatch	31.1	43.5	52.3	60.8	74.7	13.0

(b) Targeted scenario						
No. of configuration	ASR					BWO
	1	2	3	4	5	
AWA	29.2	35.1	43.9	51.2	63.5	29.1
Acup3	24.7	28.1	33.6	38.1	40.1	21.8
RLpatch	5.2	13.2	20.1	28.6	35.7	20.6

(a) Untargeted scenario (b) Targeted scenario

Robustness to Adversarial Training. As mentioned in C3, the attackers may have the ability to retrain their WF attack models by generating perturbed traffic traces using the same defense models. However, the configuration parameters of the defense models are not accessible to the attackers. To penetrate the defense, the attackers could set different variants of configuration parameters to conduct adversarial training. Specifically, the attackers can retrain their WF models based on the perturbation traces generated using different configuration parameters.

We conduct an experiment to simulate adversarial training and test the ASR of defense methods against DF attacks. we vary the configuration parameters to obtain various perturbation traces for adversarial training. Since WTF-PAD, Palette and Dolos may introduce additional bursts, we focus on AWA, KimeraPAD, Acup3, Minipatch, and RLpatch for this experiment. Specifically, in the targeted scenario, we randomly select the cluster head for Acup3 and change the target label for AWA and RLpatch. In the untargeted scenario, we use different random seeds for Minipatch and KimeraPAD, while selecting different disturbance factors for RLpatch. For the testing set, we apply a unique configuration parameter of the defense method to generate a perturbed testing dataset. For the attackers' training datasets, we apply multiple configuration parameters to generate multiple perturbed datasets, which are used for retraining the attackers' target WF model. We evaluate the ASR and BWO when the attackers' target WF model is trained with varying numbers of perturbed datasets. The experimental results are shown in Table IV.

We can observe that the ASR in the targeted scenario is generally higher than that in the untargeted scenario. This result indicates that changing the target label could increase the randomness of perturbation, thereby enhancing robustness against adversarial training. As the number of configuration variants used for adversarial training increases, it is intuitive to find an increasing trend in ASR. Since more configuration variants usually indicate more diverse perturbation traces for model retraining, the attackers are more likely to learn the perturbation pattern that is possibly employed by the defenders. Moreover, we find that the ASR of Minipatch exceeds 96.1%, exhibiting the poorest robustness when the number of configuration variants exceeds 2. This is because attackers can still generate similar perturbation traces as defenders even when using different random seeds to bypass the defense

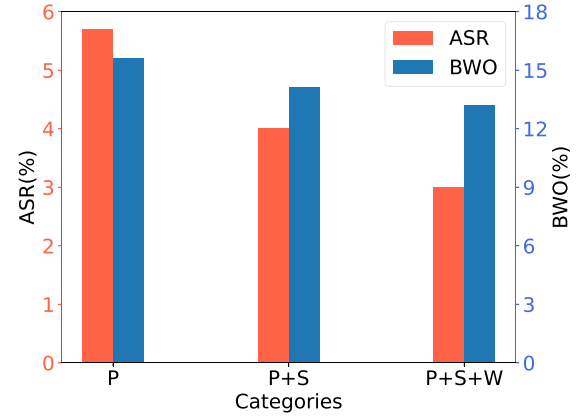


Fig. 7. ASR and BWO by varying c , ϵ and ρ .

of Minipatch. Our results show that RLpatch achieves lower ASR at a lower BWO than the evaluated methods in both the targeted and untargeted scenarios, demonstrating a more efficient overhead and effective defense performance. Therefore, RLpatch can effectively mislead the attackers' classifier while incurring minimal additional communication overhead. The attackers can hardly select the same target label or the same disturbance factor as the defenders, making it challenging to generate the same perturbation traces. Although the attackers may penetrate our defense by introducing enough configuration variants, excessive configuration variants significantly escalate the cost of the attack, as attackers need to expend more time and resources to collect and process the traffic traces for model training. Consequently, RLpatch still presents a barrier to WF attackers.

C. Ablation Study

Performance on Different Surrogate Models. To evaluate the impact of different surrogate models on RLpatch, we compare the performance for a single surrogate model to that for multiple surrogate models in Siranam Dataset. We choose one of the WF models as the target WF model and test the ASR when the perturbations, generated by either a single or multiple surrogate models, are applied to the testing dataset. Table V presents the defense performance under different target WF models. We find that generating perturbations using multiple surrogate models generally achieves better defense performance than using a single surrogate model. Due to the diversity of models, different WF attack models may have different decision boundaries and sensitive regions. The sensitive regions identified by multiple WF models are more likely critical for the classification decisions of WF target model than those identified by a single model, leading to a lower ASR. However, these regions may need to inject more perturbations to deceive the multiple WF attack models, which causes a higher bandwidth overhead, but still within an acceptable range (below 15%). Moreover, we also find that complex models, such as VarCNN and TF, are more resistant to RLpatch. In contrast, the simple model AWF can be easily defeated even when using a single surrogate model.

TABLE V
PERFORMANCE COMPARISON AMONG DIFFERENT SURROGATE MODELS

Surrogate model	Target model											
	DF(96.7,96.3) ¹			VarCNN(95.8,96.7)			AWF(87.9,91.7)			TF(94.0,94.1)		
	BWO	ASR	ANQ	BWO	ASR	ANQ	BWO	ASR	ANQ	BWO	ASR	ANQ
DF	—	—	—	7.5	19.2	7.3	5.2	7.2	5.6	7.3	19.6	9.6
VarCNN	9.6	8.9	8.1	—	—	—	9.2	6.2	6.1	8.2	17.8	9.9
AWF	5.9	11.4	6.7	6.1	25.4	9.9	—	—	—	6.1	26.0	9.3
TF	10.0	7.9	8.0	8.9	15.4	12.2	9.8	4.6	5.9	—	—	—
Multiple	13.2	2.7	6.6	11.7	7.4	9.6	13.7	1.8	5.5	11.6	7.5	8.4

¹ The former number represents the recognition accuracy of surrogate model, while the later number represents the recognition accuracy of attackers' model.

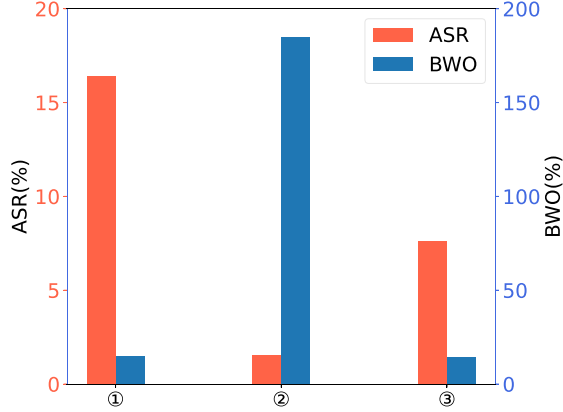


Fig. 8. ASR and BWO of different sensitive regions. ①: the sensitive regions of a randomly selected trace. ②: the sensitive regions of all traces ③: the sensitive regions aggregated by RLpatch.

Performance on Different Policy. To verify the effectiveness of each policy, we conduct experiments by adding each component individually. We select the DF as the target WF attack model to defeat in an untargeted scenario. First, we assign the same model weights to each of multiple models and set the perturbation step size to 0.2 to test the defense performance when applying only the position policy c . Next, we adjust the model weight parameter ρ according to feature maps while keeping the perturbation step size constant. Finally, we incorporate the perturbation policy ϵ into the overall learning process. The results in Fig. 7 show that when only position policy is applied, the ASR is 5.7% with a BWO of 15.1%. Both the ASR and BWO decrease after incorporating the model weight parameter ρ for adjustment, indicating that adjusting ρ effectively enhances defense performance. With the addition of the perturbation magnitude policy, we observe a further reduction in both the ASR and BWO, underscoring the importance of the perturbation magnitude ϵ to the overall defense performance.

Performance for different sensitive regions. To verify the effectiveness of the perturbation aggregation method, we conduct experiments by covering different sensitive regions to apply perturbations. For each website, we add perturbations by ① covering the sensitive regions of a randomly selected trace; ② covering the sensitive regions of all traces; ③ covering the sensitive regions aggregated by RLpatch. Specifically, the

generations of different sensitive regions are as follows: ① We randomly select a trace from each website's traces and generate a common perturbation paradigm of the website using its sensitive region. ② We cover all sensitive regions of the training traces of the same website by overlaying their perturbation positions and taking the maximum perturbation magnitude at each position. ③ RLpatch aggregates the sensitive regions based on positional frequency to construct a common perturbation paradigm for each website. We apply these generated perturbation paradigms to testing dataset, which consists of previously unseen traffic traces of the same website, and evaluate their ASR and BWO against the target DF model. The comparison results are shown in Fig. 8. It can be seen that although adding perturbation on all sensitive regions can significantly reduce the ASR to 1.5%, it may lead to an unacceptable BWO of 184.6%. In contrast, covering the sensitive regions of a single trace yields poor defense performance, achieving a much higher ASR of 16.4%, but a lower BWO of 14.7%. Due to variations in network environments and user behavior, the sensitive regions often differ across traces of the same website, and perturbations tailored to one trace may not generalize effectively. Whereas, RLpatch slightly increases the ASR to 7.6%, but significantly reduces the BWO to 14.1%, demonstrating its effectiveness and practicality for real-world deployment. By retaining only some key sensitive regions, RLpatch achieves strong defense performance across diverse traffic traces of the same website while maintaining a low BWO.

D. Effects of Hyperparameters

We investigate the impact of key parameters on RLpatch, including the disturbance factor η , learning rate α and threshold λ .

Disturbance factor η . It controls the perturbation generator to create diverse perturbations for different clients. To evaluate the impact of the disturbance factor on the defense performance, we randomly generate five different vectors for η . We use DF as the target model and run RLpatch with each η to generate corresponding perturbations. The experimental results are presented in Table VI. We can observe that the ASRs are all below 5% while the BWOs are all below 13.2% for all disturbance factors. Moreover, all the ASRs and BWOs vary by less than 2% across all disturbance factors. This variation indicates that RLpatch's defense performance is

TABLE VI
THE IMPACT OF DISTURBANCE FACTORS AND LEARNING RATE

Metric	Disturbance Factor				
	η_1	η_2	η_3	η_4	η_5
ASR (%)	4.4	2.7	3.1	4.6	3.4
BWO (%)	11.3	13.2	12.6	11.2	12.1
Metric	Learning rate				
	10^{-4}	5×10^{-4}	10^{-3}	5×10^{-3}	10^{-2}
ASR (%)	4	3.1	2.7	4.4	3.8
BWO (%)	11.6	12.1	13.2	11.9	12.4

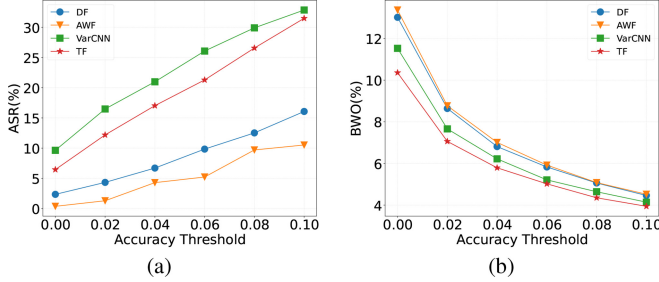


Fig. 9. ASR and BWO vary with threshold λ .

largely unaffected by choice of disturbance factor η , thereby demonstrating its robustness and reducing the need for fine-grained hyperparameter tuning.

Learning rate α . The learning rate α is weight coefficient to control the magnitude of the policy gradient change. To explore the effects of different learning rates on the defense performance, we design five sets of experiments with α ranging from 10^{-4} to 10^{-2} . We use DF as the target model and run RLpatch with each α to generate corresponding perturbations. The corresponding perturbation results against the DF attacks are presented in Table VI. As we can observe, by setting the learning rate to be 10^{-3} , RLpatch achieves the testing best defense performance against the DF model. Moreover, the results also reveal that RLpatch is well resistant to each attack model across different learning rates, since the overall variation in performance remains relatively modest. This indicates that RLpatch is not overly sensitive to the exact choice of learning rate α and can maintain effective defense within a reasonable range.

Threshold λ . It determines whether a candidate perturbation should be retained based on its contribution to reducing the ASRs of surrogate models during the perturbation aggregation process. A higher λ results in more stringent filtering, allowing only perturbations in sensitive regions with stronger defense effects to be preserved. We fix the other parameters of RLpatch and increase λ from 0 to 0.1. We observe that RLpatch demonstrates an escalation in ASR and a decline in BWO across all models as λ increases. As the threshold λ is set higher, RLpatch is more likely to retain only the perturbation with strong effects on defense performance, while dropping the perturbations with weak effects, which may lead to a decline in ASR. In contrast, the corresponding BWO diminishes as λ rises due to fewer retained perturbations. Therefore, an appropriate threshold should be selected to balance ASR and BWO.

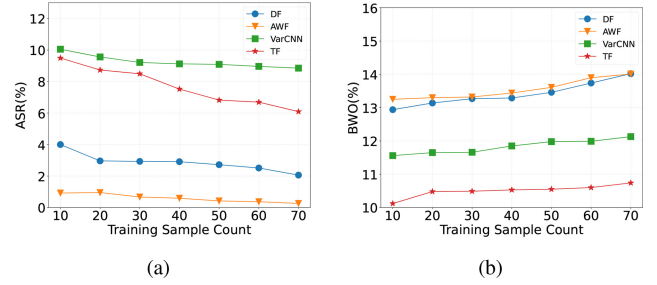


Fig. 10. Performance for different number of training samples.

TABLE VII
PERFORMANCE COMPARISON BETWEEN UNIDIRECTIONAL AND BIDIRECTIONAL PADDING

Metric	ASR(%)	BWO(%)	ANQ
Outgoing-Padding	5.7	14.8	8.6
Incoming-Padding	35.4	19.2	29.0
Bidirectional-Padding	2.7	13.2	6.6

E. Implementation Results

We further discuss additional properties of our defense when deployed in real scenarios, including the effects of unidirectional padding, training sample counts and open-world setting.

Performance on Unidirectional Padding. We conduct experiments to assess the viability of RLpatch's padding deployment strategy. When implementing unidirectional padding, we adjust the corresponding constraints in the process of crafting adversarial perturbations. Specifically, in the outgoing-padding scenario, the values at even positions of the adversarial perturbation sequence are set to zero, while in the incoming-padding scenario, the values at odd positions are set to zero. We choose the DF as the target model for evaluation and show the results in Table VII. The results indicate that the ASR of unidirectional padding is higher than that of bidirectional padding, especially for incoming-padding. In contrary, we find that padding only in the outgoing direction yields results similar to the bidirectional padding, indicating that the outgoing-padding primarily contributes to the perturbation effect. The above results demonstrate that RLpatch can still work well when only deploying RLpatch on the user side.

Performance on Data Efficiency. We investigate the impact of the number of training samples during the reinforcement learning process on defense performance, which may directly affect the training time and the effectiveness of adversarial perturbations. We use the DF, AWF, VarCNN, TF as target WF models to defeat respectively, and run RLpatch with different numbers of training samples. The experimental results are shown in Fig. 10. We can observe that the ASR of RLpatch decreases while the BWO increases as the number of training samples increases from 10 to 70. This occurs because a larger training sample size allows RLpatch to identify sensitive regions more accurately for crafting adversarial perturbations, thereby reducing the ASR. Moreover, since the same website may produce different traffic traces, RLpatch requires to inject additional dummy packets for modification. This further

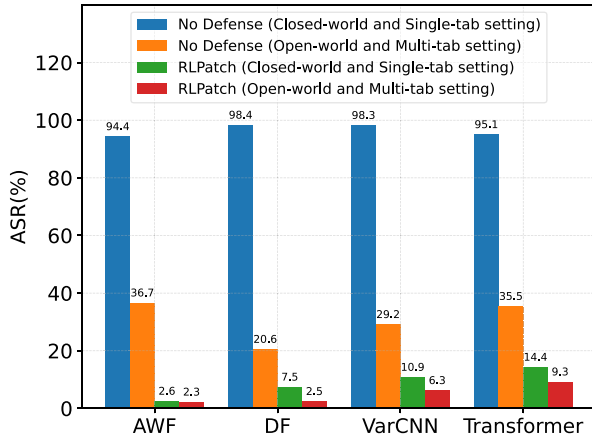


Fig. 11. ASR Comparison under closed-world & single-tab setting and open-world & multi-tab setting.

misleads the attacker's WF model but inevitably leads to an increase in bandwidth overhead.

Performance on Real-world Scenarios. In the above evaluation experiment, we have demonstrated that RLPatch outperforms state-of-the-art defenses in the closed-world and single-tab setting, where WF attacks are trained as multi-class classification models over a fixed set of monitored websites. Although this setting is favorable to attackers and represents a challenging scenario for defenders, it does not fully reflect the complexity of real-world usage patterns. To further validate the practicality and robustness of RLPatch, we evaluate its performance in a real-world scenario, where users may visit websites outside the monitored set and operate multiple browser tabs simultaneously, that is, on the open-world and multi-tab setting. Specifically, we select 50 websites from the Rimmer dataset as the monitored set and another 50 websites as the unmonitored set. We first generate the perturbed traces from the monitored set based on the original traces and the corresponding common perturbation paradigm. We then construct a multi-tab trace by merging a perturbed trace from the monitored set with an unperturbed trace from the unmonitored set. In this way, we create a testing dataset that simulates the open-world and multi-tab browsing behavior. Similarly, we generate a new testing dataset without perturbations to serve as a baseline for comparison.

The experimental results are shown in Fig. 11. Without any defense, WF models can achieve high ASR in the closed-world and single-tab setting, ranging from 94.4% of AWF to 98.4% of DF. However, in the open-world and multi-tab setting, the average ASR drops to 30.5% across four different WF attacks, indicating that concurrent browsing and visits to unmonitored websites inherently increase the difficulty of WF attacks. This is because multi-tab traffic traces become interwoven, generating mutual noise among website sessions. Such overlapping traffic patterns obscure site-specific features, making it challenging for WF models to accurately extract clean and distinctive characteristics. After applying RLPatch, we observe a substantial reduction in ASR across all WF attack models. In the closed-world and single-tab setting, RLPatch reduces the average ASR to 8.9%. More importantly, in the

open-world and multi-tab setting, RLPatch further lowers the average ASR to just 6.1%. These results demonstrate that RLPatch not only excels in closed-world and single-tab setting but also maintains strong defense performance under complex real-world conditions.

VII. DISCUSSION

A. Practical Deployment

The defense requires the cooperation of both sides of the network connection (for Tor, the client and the entry node). Here, RLPatch assumes that a pluggable transport (PT) proxy is deployed at both the client and the Tor entry node, with synchronized perturbation paradigms to ensure that dummy packets are injected at both ends. If potential synchronization failures occur, one of the PT proxies cannot filter out the dummy packets, leading to connection error.

However, two practical challenges remain to be addressed. The first challenge is to determine how to correctly remove dummy packets to recover normal communication. In our work, since we add the dummy packets at the end of each burst, the receiving side can discard them based on the shared perturbation paradigm. However, if the transmission path changes during delivery, packet reordering may occur. In such cases, blindly removing packets at the end of a burst could result in discarding legitimate data, triggering retransmissions and introducing additional delays. As a supplement, we can employ a link-padding layer between the transport and application layers to mark the dummy packets using a defined flag in the protocol header. This flag can assist in the identification and removal of dummy packets, thereby reducing the risk of misinterpretation and ensuring robust delivery even under unstable network conditions. The second challenge is to determine when to start injecting dummy packets, as the PT proxy cannot know the length of the current burst in advance. If the client/server stops sending real packets for a time interval, the current burst is considered to be ended and we can start inserting dummy packets.

B. Training and Deployment Cost Analysis

We evaluate the computational costs associated with training and implementation in real-world settings. Firstly, defenders need to collect traffic traces to generate perturbations. Each traffic trace is approximately 40 KB in size on average. Assuming that the defenders intend to protect 100 websites and collect 30 traces for each website, the total storage requirement would be approximately $40 \text{ KB} \times 100 \times 30 = 120 \text{ MB}$.

Afterwards, we measure the average training time, GPU memory usage, and CPU utilization during both the training and implementation stages. When using a single CPU core, RLPatch requires approximately 15 minutes of training time and 3000 MB of GPU memory on the Sirinam dataset to generate a common perturbation paradigm for one monitored website. On the Rimmer dataset, the training time increases slightly to around 20 minutes with similar GPU memory. Since training is performed offline, this time cost is considered acceptable for practical use.

Finally, defenders need to deploy the generated common perturbation paradigms to both the PT proxy on the client side and the Tor entry node. Each perturbation paradigm file is approximately 6 KB in size per website. Their lightweight nature enables rapid updates and scalability, making it feasible to support a large number of monitored websites without imposing significant storage or transmission burdens on the defenders' infrastructure. Consequently, the deployment of RLpatch remains practical and manageable even in resource-constrained environments.

C. Limitation

While most state-of-the-art WF attacks operate on packet direction sequences, some recent works have demonstrated that comparable classification performance can also be achieved using packet timestamps. However, our current defense mechanism is not designed to address attacks that rely primarily on timing features. RLpatch focuses on modifying direction-based bursts by injecting dummy packets, but does not manipulate packet timing characteristics such as inter-packet delays or burst intervals. To defend against timing-based WF attacks, future extensions of RLpatch may jointly optimize direction and time features in future work. Moreover, RLpatch focuses on adding tiny perturbation to mislead DNN-based WF attacks. Non-DNN traffic analysis techniques that use statistical features (e.g., packet counts, traffic volumes, and connection duration) may not be protected by our defense.

VIII. CONCLUSION

This paper introduces RLpatch, a robust low-overhead defense method against WF attacks based on RL, which focuses on generating perturbation within sensitive regions to reduce the bandwidth overhead while keeping the effectiveness to mislead the WF attack models. RLpatch employs a novel RL-based adversarial perturbation generation method to identify the common sensitive regions of various surrogate models and adjust perturbation to fit the traffic trace by a query WF model for the feedback. By aggregating the perturbations from the same website based on positional frequency, RLpatch can generate a common perturbation paradigm to prevent the traffic traces of the website from being accurately identified by attackers. Experiments demonstrate that RLpatch can outperform other baseline WF defense methods on bandwidth overhead and defense performance against SOTA WF attacks. For future work, we will focus on designing an adaptive defense mechanism to ensure that the defense remains effective even as Web traffic trace changes over time.

REFERENCES

- [1] D. Harborth, S. Pape, and K. Rannenberg, "Explaining the technology use behavior of privacy-enhancing technologies: The case of Tor and JonDonym," in *Proc. Privacy Enhanc. Technol. (PETs)*, 2020, pp. 111–128.
- [2] X. Deng, R. Zhao, Y. Wang, M. Zhan, Z. Xue, and Y. Wang, "Countmamba: A generalized website fingerprinting attack via coarse-grained representation and fine-grained prediction," in *Proc. IEEE Symp. Security Privacy (SP)*, 2025, pp. 1363–1381.
- [3] A. Maghsoudlou, L. Vermeulen, I. Poese, and O. Gasser, "Characterizing the VPN ecosystem in the wild," in *Proc. Conf. Passive Active Meas.*, 2023, pp. 18–45.
- [4] P. Krämer et al., "ProFi: Scalable and efficient website fingerprinting," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 1271–1286, Feb. 2024.
- [5] B. Gao, W. Liu, G. Liu, F. Nie, and J. Huang, "Multi-level resource-coherented graph learning for website fingerprinting attacks," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 693–708, 2025.
- [6] M. Juárez, M. Imani, M. Perry, C. Díaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. 21st ESORICS*, 2016, pp. 27–46.
- [7] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 717–734.
- [8] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. Security Privacy (SP)*, 2012, pp. 332–346.
- [9] J. Huang, W. Liu, G. Liu, B. Gao, and F. Nie, "WF-A2D: Enhancing privacy with asymmetric adversarial defense against website fingerprinting," *IEEE Trans. Inf. Forensic Security*, vol. 20, pp. 4739–4754, 2025.
- [10] M. Jiang, B. Cui, J. Fu, T. Wang, L. Yao, and B. K. Bhargava, "RUDOLF: An efficient and adaptive defense approach against website fingerprinting attacks based on soft actor-critic algorithm," *IEEE Trans. Inf. Forensic Security*, vol. 19, pp. 7794–7809, 2024.
- [11] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2014, pp. 143–157.
- [12] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2016, pp. 1187–1203.
- [13] A. Panchenko et al., "Website fingerprinting at Internet scale," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2016, Art. no. 23477.
- [14] D. Herrmann, R. Wendolsky, H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier," in *Proc. ACM Cloud Comput. Secur. Workshop (CCSW)*, 2019, pp. 31–42.
- [15] V. Rimmer, D. Preuveneers, M. Juárez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2017, pp. 1–15.
- [16] P. Sirinam, M. Imani, M. Juárez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 1928–1943.
- [17] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," in *Proc. Privacy Enhanc. Technol. (PETs)*, 2019, pp. 292–310.
- [18] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 607–624.
- [19] Y. Yuan, Y. Du, and G. Cheng, "Class incremental website fingerprinting attack based on dynamic expansion architecture," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 1955–1971, Apr. 2025.
- [20] H. Zou, J. Su, Z. Wei, S. Chen, C. Yang, and M. Chen, "Toward an effective few-shot website fingerprinting attack with quadruplet networks and deep local fingerprinting features," *IEEE Trans. Dependable Secur. Comput.*, early access, Apr. 22, 2025, doi: 10.1109/TDSC.2025.3563389.
- [21] W. Meng, C. Ma, M. Ding, C. Ge, Y. Qian, and T. Xiang, "Beyond single tabs: A transformative few-shot approach to multi-tab website fingerprinting attacks," in *Proc. ACM Web Conf.*, 2025, pp. 1068–1077.
- [22] X. Cai, R. Nithyanand, and R. Johnson, "Cs-BuFLO: A congestion sensitive website fingerprinting defense," in *Proc. ACM Workshop Privacy Electron Soc.*, 2014, pp. 121–130.
- [23] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2014, pp. 227–238.
- [24] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 1375–1390.
- [25] M. Shen et al., "Real-time website fingerprinting defense via traffic cluster anonymization," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2024, pp. 3238–3256.
- [26] M. S. Rahman, M. Imani, N. Mathews, and M. Wright, "Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1594–1609, 2021.

- [27] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2705–2722.
- [28] S. Shan, A. N. Bhagoji, H. Zheng, and B. Y. Zhao, "Patch-based defenses against Web fingerprinting attacks," in *Proc. ACM Workshop Artif. Intell. Secur.*, 2021, pp. 97–109.
- [29] A. M. Sadeghzadeh, B. Tajali, and R. Jalili, "AWA: Adversarial website adaptation," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3109–3122, 2021.
- [30] D. Li, Y. Zhu, M. Chen, and J. Wang, "Minipatch: Undermining DNN-based website fingerprinting with adversarial patches," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2437–2451, 2022.
- [31] L. Qiao, B. Wu, S. Yin, H. Li, W. Yuan, and X. Luo, "Resisting DNN-based website fingerprinting attacks enhanced by adversarial training," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5375–5386, 2023.
- [32] M. Jiang, B. Cui, J. Fu, T. Wang, and Z. Wang, "KimeraPAD: A novel low-overhead real-time defense against website fingerprinting attacks based on deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2944–2961, Jun. 2024.
- [33] C. Hou, G. Gou, J. Shi, P. Fu, and G. Xiong, "WF-GAN: Fighting back against website fingerprinting attack using adversarial learning," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, 2020, pp. 1–7.
- [34] M. H. M. Soleimani, M. Mansoorizadeh, and M. Nassiri, "Real-time identification of three Tor pluggable transports using machine learning techniques," *J. Supercomput.*, vol. 74, no. 10, pp. 4910–4927, 2018.
- [35] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2921–2929.
- [36] Y. Dong et al., "Boosting adversarial attacks with momentum," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 9185–9193.
- [37] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Med. Image Comput. Comput.-Assist. Interv.*, 2015, pp. 234–241.
- [38] D. Zhou, S. Gao, and F. Long, "Research on intrusion detection based on Markov chain monte carlo method," in *Proc. Int. Conf. Cloud Comput. Big Data Appl. Softw. Eng.*, 2023, pp. 184–189.
- [39] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-Transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.



Yuxing Wei received the bachelor's degree in computer science and technology from the Guangxi University for Nationalities in July 2013. He is currently affiliated with the Information Center of Guangxi Zhuang Autonomous Region. His research interests include cloud computing, government network systems, and network and data security.



Dong Jin received the B.S. degree in automation from the Hefei University of Technology, Hefei, China, in 2018, and the Ph.D. degree in electronic information from the University of Science and Technology of China, Hefei, in 2023. He is currently an Associate Research Fellow with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei. His research interests include network optimization, industrial Internet, and AI-aided network intrusion detection.



Xiaobin Tan (Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China, China, in 1996 and 2003, respectively, where he is currently an Associate Professor with the Department of Automation, School of Information Science and Technology. His research interests include future Internet architecture, information security, and multimedia communication.



Shuangwu Chen (Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2011 and 2016, respectively, where he is currently an Associate Professor with the Department of Automation, School of Information Science and Technology. His research interests include multimedia communications, future network, and stochastic optimization.



Xiaofeng Jiang received the B.S. and Ph.D. degrees in information science and technology from the University of Science and Technology of China, Hefei, China, in 2008 and 2013, respectively, where he is currently an Associate Professor with the School of Information Science and Technology. His recent research interests include discrete-event dynamic system, tensor analysis and big data, future network, and cognitive communications.



Siyang Chen received the B.S. degree in information science and technology from the University of Science and Technology of China, Hefei, China, in 2019, where he is currently pursuing the Ph.D. degree. His research interests include network security and traffic analysis.



Jian Yang (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2001 and 2006, respectively, where he is currently a Professor with the School of Information Science and Technology. His research interests include future networks, distributed system design, modeling and optimization, multimedia over wired and wireless networks, and stochastic optimization.